part2assesement

2025-10-26

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com (http://rmarkdown.rstudio.com).

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this: **Question 1**

```
library(ggplot2)
# Load recent seasons
pbp <- load_pbp(2023)
# Filter for 4th down plays only
fourth_down <- pbp %>%
filter(down == 4, !is.na(yardline_100)) %>%
mutate(go_for_it = ifelse(play_type == "run" | play_type == "pass", 1, 0))
# Fit a linear regression: probability of going for it ~ yardline_100
model <- lm(go_for_it ~ yardline_100, data = fourth_down)
# Summarize model
summary(model)
```

```
##
## Call:
## lm(formula = go_for_it ~ yardline_100, data = fourth_down)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.34923 -0.22124 -0.13367 -0.07304  0.98085
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.3526019  0.0128050   27.54   <2e-16 ***
## yardline_100 -0.0033682  0.0002318  -14.53   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3802 on 4480 degrees of freedom
##   (8 observations deleted due to missingness)
## Multiple R-squared:  0.045,  Adjusted R-squared:  0.04479
## F-statistic: 211.1 on 1 and 4480 DF,  p-value: < 2.2e-16
```

Including Plots

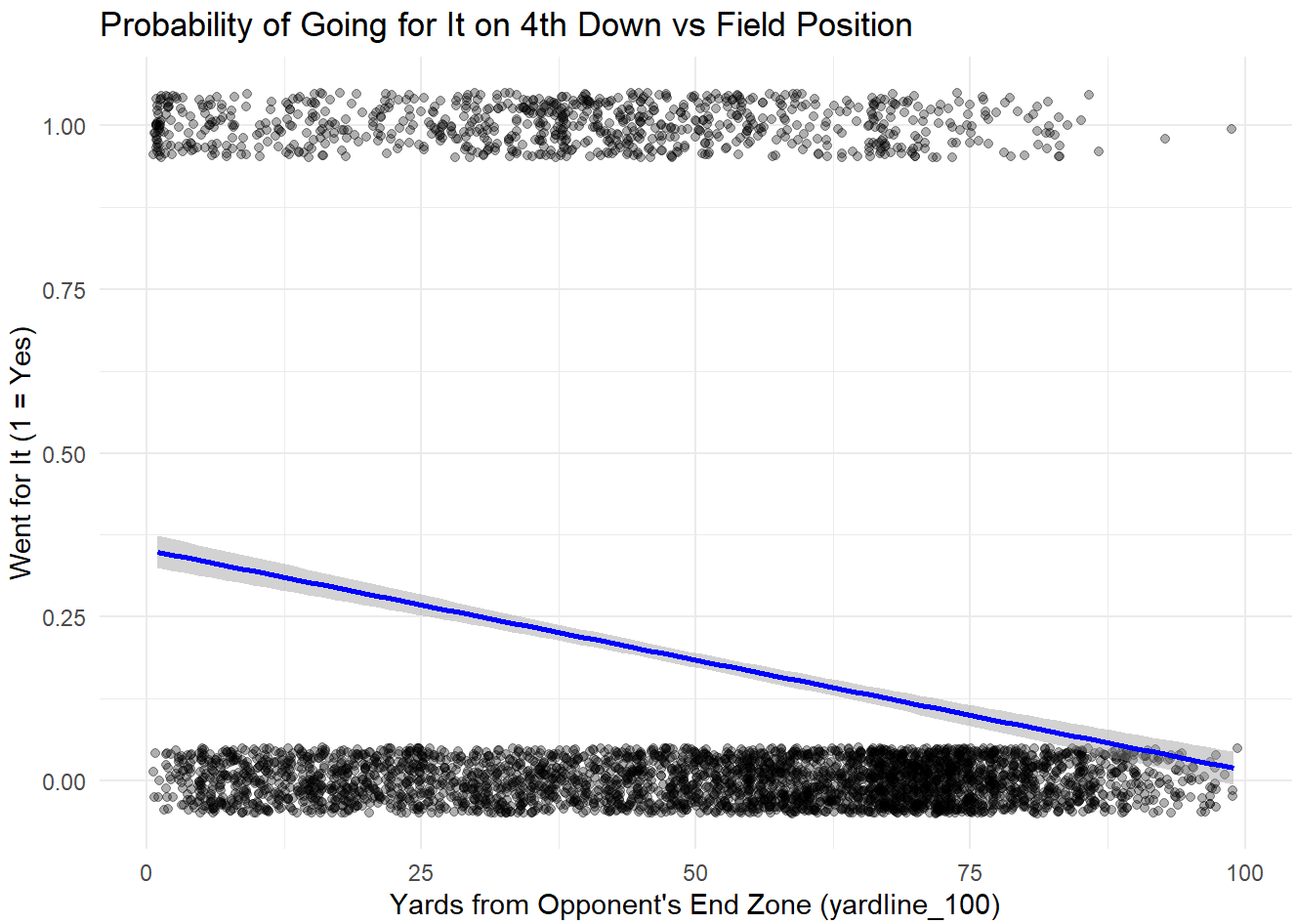
You can also embed plots, for example:

```
# Graph data with fitted line
ggplot(fourth_down, aes(x = yardline_100, y = go_for_it)) +
geom_jitter(alpha = 0.3, height = 0.05) +
geom_smooth(method = "lm", color = "blue") +
labs(
title = "Probability of Going for It on 4th Down vs Field Position",
x = "Yards from Opponent's End Zone (yardline_100)",
y = "Went for It (1 = Yes)"
) +
theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 8 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 8 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot. **Question 2**

```
# Compute influence metrics directly from the model
leverage <- hatvalues(model)
cooks <- cooks.distance(model)
res <- resid(model)
# Get the data actually used in the model
used_data <- model.frame(model)
# Make sure everything aligns
length(leverage); length(cooks); length(res); nrow(used_data)
```

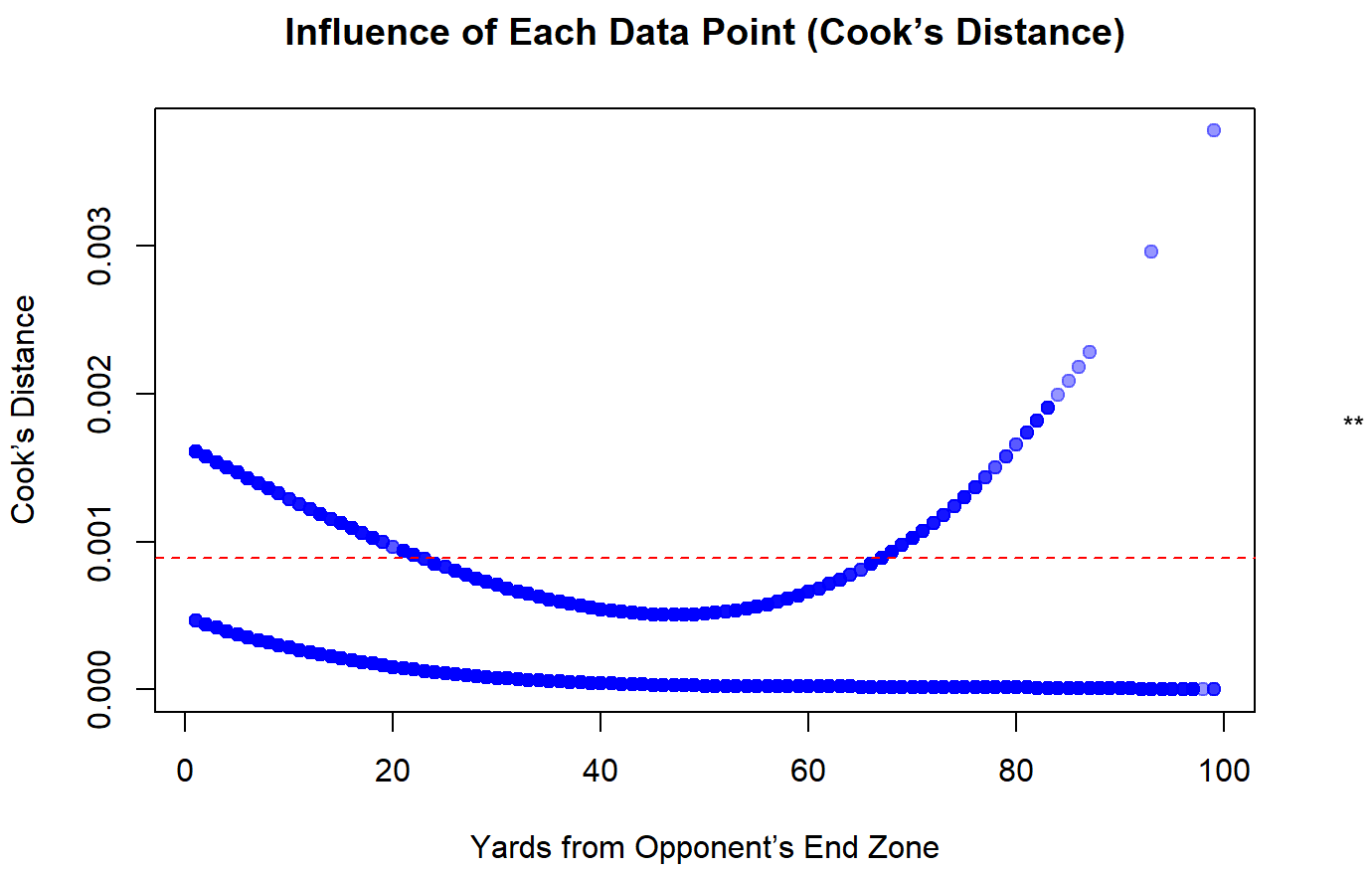
```
## [1] 4482
```

```
## [1] 4482
```

```
## [1] 4482
```

```
## [1] 4482
```

```
# Combine all into one data frame
influence_data <- data.frame(
yardline_100 = used_data$yardline_100,
leverage = leverage,
cooks = cooks,
resid = res
)
# Plot Cook’s Distance correctly
plot(influence_data$yardline_100, influence_data$cooks,
main = "Influence of Each Data Point (Cook’s Distance)",
xlab = "Yards from Opponent’s End Zone",
ylab = "Cook’s Distance",
pch = 19, col = rgb(0, 0, 1, 0.4))
# Add cutoff line
abline(h = 4 / length(cooks), col = "red", lty = 2)
```



The centroid of the data - around x = 50, where the Cook's Distance reaches its minimum (nearly 0).Points near the center of the x-range have minimal leverage and therefore minimal influence on the regression line's slope.To have large influence on the estimated slope, add a data point at the extremes of the x-range. These points are near x = 0 or x = 100. The plot shows Cook's Distance increasing dramatically at both ends, with the highest values at the far right (x ≈ 100, Cook's D ≈ 0.003-0.004).**

Question 3

```
# Make sure you have your filtered dataset
fourth_down <- pbp %>%
filter(down == 4, !is.na(yardline_100), !is.na(posteam_type)) %>%
mutate(go_for_it = ifelse(play_type %in% c("run", "pass"), 1, 0))
# Separate models for Home vs Away
model_home <- lm(go_for_it ~ yardline_100, data = fourth_down, subset = posteam_type == "home")
model_away <- lm(go_for_it ~ yardline_100, data = fourth_down, subset = posteam_type == "away")
summary(model_home)
```

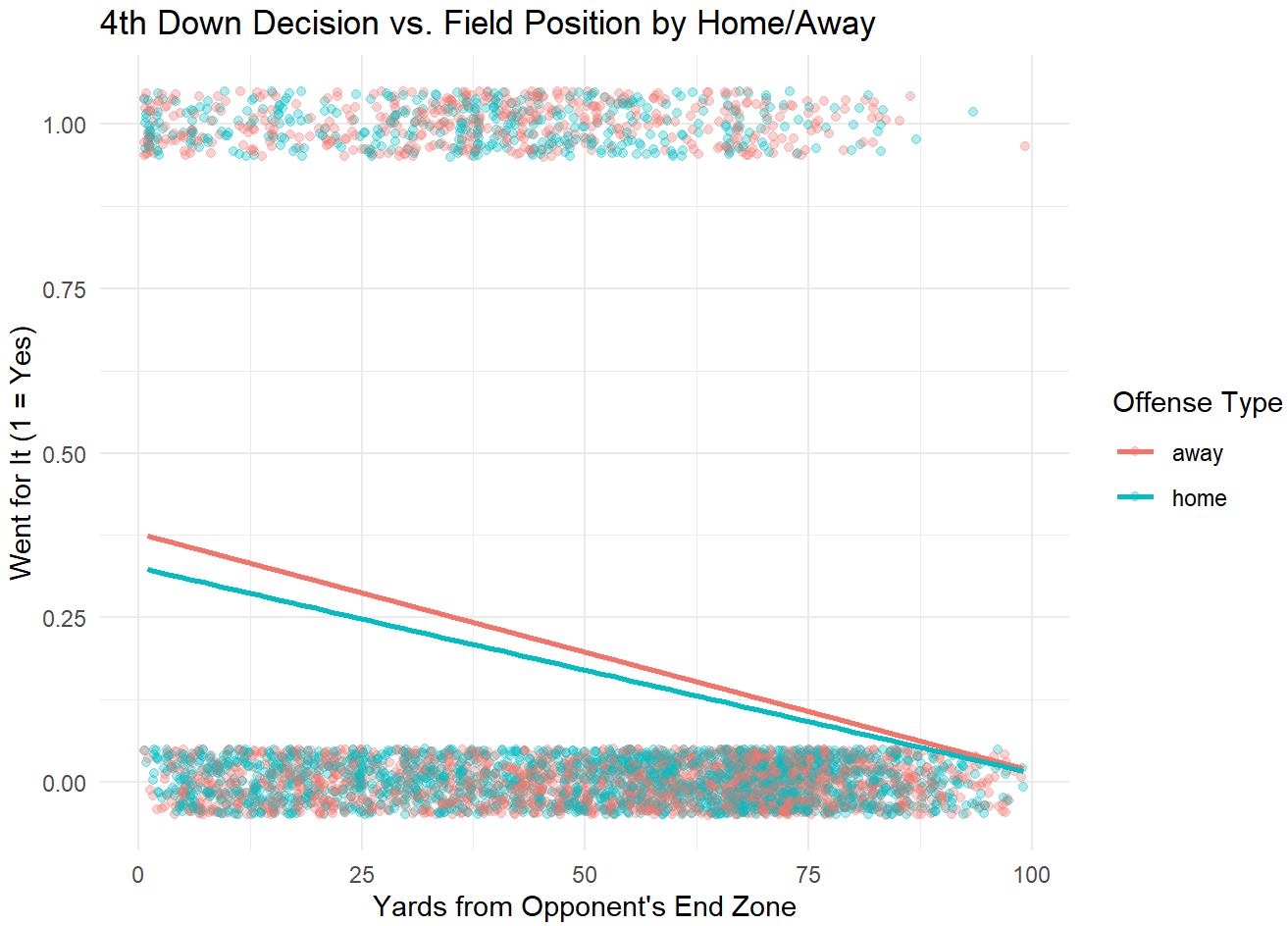
```
##
## Call:
## lm(formula = go_for_it ~ yardline_100, data = fourth_down, subset = posteam_type ==
##     "home")
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -0.3229 -0.2137 -0.1294 -0.0764  0.9642
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.326000   0.017652  18.468   <2e-16 ***
## yardline_100 -0.003120   0.000323  -9.658   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3711 on 2205 degrees of freedom
## Multiple R-squared:  0.04059,    Adjusted R-squared:  0.04015
## F-statistic: 93.28 on 1 and 2205 DF,  p-value: < 2.2e-16
```

```
summary(model_away)
```

```
##
## Call:
## lm(formula = go_for_it ~ yardline_100, data = fourth_down, subset = posteam_type ==
##     "away")
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -0.3746 -0.2336 -0.1359 -0.0708  0.9798
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.3782600  0.0184796   20.47   <2e-16 ***
## yardline_100 -0.0036172  0.0003315  -10.91   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.388 on 2281 degrees of freedom
## Multiple R-squared:  0.04962,    Adjusted R-squared:  0.0492
## F-statistic: 119.1 on 1 and 2281 DF,  p-value: < 2.2e-16
```

```
ggplot(fourth_down, aes(x = yardline_100, y = go_for_it, color = posteam_type)) +
geom_jitter(alpha = 0.3, height = 0.05) +
geom_smooth(method = "lm", se = FALSE) +
labs(
title = "4th Down Decision vs. Field Position by Home/Away",
x = "Yards from Opponent's End Zone",
y = "Went for It (1 = Yes)",
color = "Offense Type"
) +
theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Question 4

```
#Question 4


# estimate_n_from_mads.R
# Back out sample size n from posterior MADs and sigma for a simple regression.
# GIVEN (from the output)
MAD_b0 <- 1.4 # Intercept MAD
MAD_b1 <- 0.1 # Slope MAD
sigma <- 11.6 # Residual SD (sigma), median
# Convert MAD -> SD assuming ~Normal posterior: SD ≈ 1.4826 * MAD
SD_from_MAD <- function(mad) 1.4826 * mad
SE_b1 <- SD_from_MAD(MAD_b1) # ≈ 0.14826
SE_b0 <- SD_from_MAD(MAD_b0) # ≈ 2.07564
cat("SE_b1 ~", round(SE_b1, 5), " | SE_b0 ~", round(SE_b0, 5), "\n")
```

```
## SE_b1 ~ 0.14826  | SE_b0 ~ 2.07564
```

```
# OLS identity: SE(b1) = sigma / sqrt(Sxx) => Sxx = (sigma / SE_b1)^2
Sxx <- (sigma / SE_b1)^2
cat("Implied Sxx ~", round(Sxx, 2), "\n")
```

```
## Implied Sxx ~ 6121.64
```

```
# Intercept SE identity: SE(b0)^2 = sigma^2 * ( 1/n + xbar^2 / Sxx )
# If x is mean-centered (xbar ≈ 0), then 1/n ≈ (SE_b0 / sigma)^2
n_est <- 1 / ((SE_b0 / sigma)^2)
cat("n estimate (assuming centered x):", n_est, " -> rounded:", round(n_est), "\n\n")
```

```
## n estimate (assuming centered x): 31.23287  -> rounded: 31
```

```
# Sanity-check via simulation
set.seed(123)
n <- round(n_est)
var_x <- Sxx / (n - 1) # since Sxx = sum (x - xbar)^2 = (n - 1)*Var(x)
mu_x <- 0 # center x
B0 <- 24.8
B1 <- 0.5
sig <- sigma
nsim <- 400
grab <- replicate(nsim, {
x <- rnorm(n, mean = mu_x, sd = sqrt(var_x))
y <- B0 + B1*x + rnorm(n, 0, sig)
s <- summary(lm(y ~ x))
c(se_b1 = coef(s)[2, "Std. Error"],
se_b0 = coef(s)[1, "Std. Error"],
sigma = s$sigma)
})
sim_medians <- apply(grab, 1, median)
targets <- c(target_se_b1 = SE_b1, target_se_b0 = SE_b0, target_sigma = sigma)
cat("Simulation medians:\n"); print(round(sim_medians, 4))
```

```
## Simulation medians:
```

```
##   se_b1   se_b0   sigma
##  0.1492  2.1024 11.4861
```

```
cat("\nTargets:\n"); print(round(targets, 4))
```

```
##
## Targets:
```

```
## target_se_b1 target_se_b0 target_sigma
##       0.1483       2.0756      11.6000
```

```
# Problem 5 (c), (d), (e)

hibbs <- read.table(
  "https://raw.githubusercontent.com/avehtari/ROS-Examples/master/ElectionsEconomy/data/hibbs.da
t",
  header = TRUE
)
stopifnot(all(c("vote","growth") %in% names(hibbs)))
y <- hibbs$vote
x <- hibbs$growth
# Expect columns named 'vote' and 'growth'
stopifnot(all(c("vote", "growth") %in% names(hibbs)))
y <- hibbs$vote
x <- hibbs$growth
#5(c) Least Absolute Deviation (LAD) vs Least Squares (LS)
# LS fit
fit_ls <- lm(y ~ x)
coef_ls <- coef(fit_ls) # (Intercept), x
# LAD objective and estimate (via Nelder–Mead)
lad_obj <- function(par) {
a <- par[1]; b <- par[2]
sum(abs(y - (a + b * x)))
}
opt_lad <- optim(par = c(coef_ls[1], coef_ls[2]),
fn = lad_obj,
method = "Nelder-Mead",
control = list(reltol = 1e-10, maxit = 10000))
coef_lad <- setNames(opt_lad$par, c("(Intercept)", "x"))
cat("\n--- 5(c) Estimates ---\n")
```

```
##
## --- 5(c) Estimates ---
```

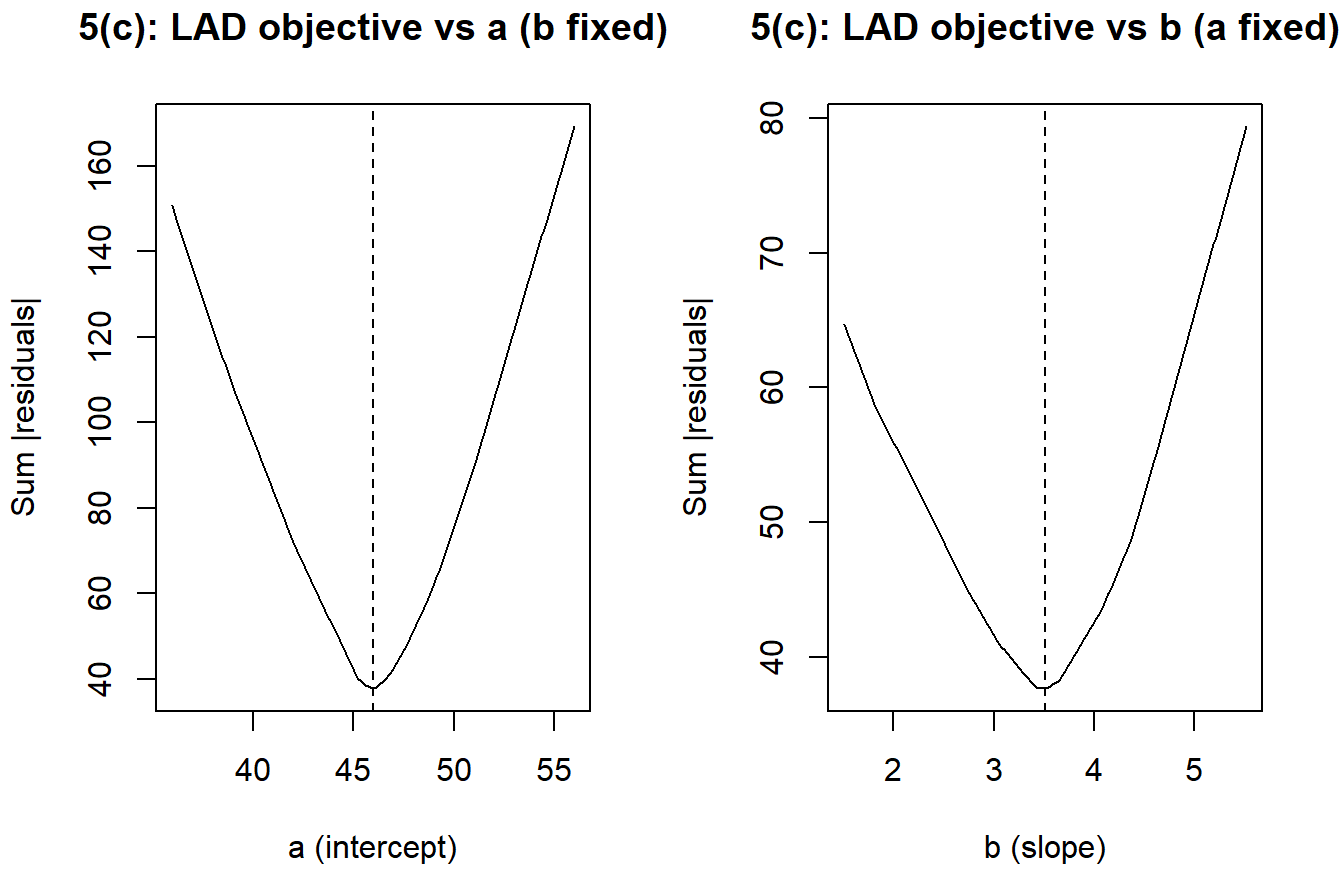
```
cat(sprintf("LS (a, b): %.3f, %.3f\n", coef_ls[1], coef_ls[2]))
```

```
## LS (a, b): 46.248, 3.061
```

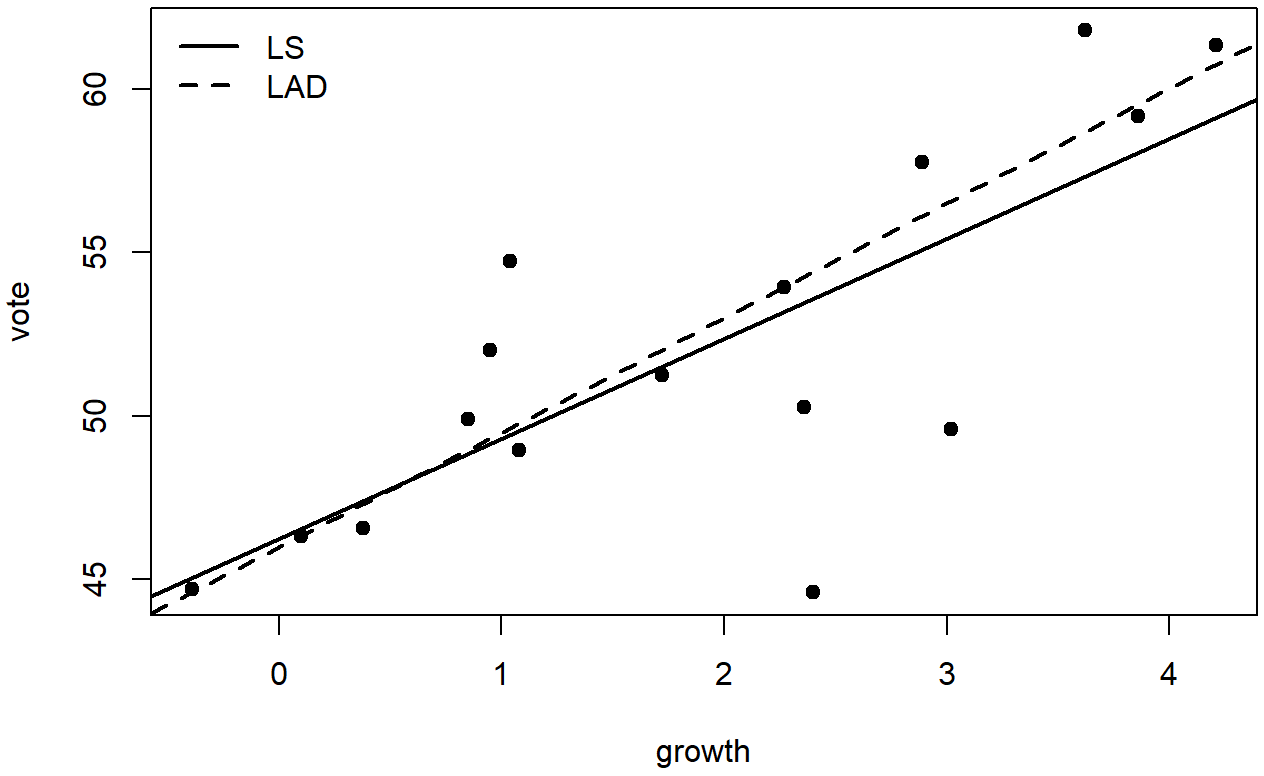
```
cat(sprintf("LAD (a, b): %.3f, %.3f\n", coef_lad[1], coef_lad[2]))
```

```
## LAD (a, b): 45.969, 3.512
```

```
# LAD objective vs a (fix b = LAD b) and vs b (fix a = LAD a)
agrid <- seq(coef_lad[1] - 10, coef_lad[1] + 10, length.out = 200)
bgrid <- seq(coef_lad[2] - 2, coef_lad[2] + 2, length.out = 200)
lad_vs_a <- sapply(agrid, function(a) lad_obj(c(a, coef_lad[2])))
lad_vs_b <- sapply(bgrid, function(b) lad_obj(c(coef_lad[1], b)))
op <- par(mfrow = c(1, 2))
plot(agrid, lad_vs_a, type = "l", xlab = "a (intercept)", ylab = "Sum |residuals|",
main = "5(c): LAD objective vs a (b fixed)")
abline(v = coef_lad[1], lty = 2)
plot(bgrid, lad_vs_b, type = "l", xlab = "b (slope)", ylab = "Sum |residuals|",
main = "5(c): LAD objective vs b (a fixed)")
abline(v = coef_lad[2], lty = 2)
```




```
par(op)
# Visual comparison: LS and LAD lines
plot(x, y, pch = 19, main = "Hibbs: LS vs LAD fits", xlab = "growth", ylab = "vote")
abline(fit_ls, lwd = 2)
abline(a = coef_lad[1], b = coef_lad[2], lwd = 2, lty = 2)
legend("topleft", legend = c("LS", "LAD"), lwd = 2, lty = c(1, 2), bty = "n")
```




```
#5(d) Construct data where LS and LAD differ a lot
set.seed(123)
n_clean <- 30
x_clean <- runif(n_clean, 0, 10)
y_clean <- 2 + 0.8 * x_clean + rnorm(n_clean, 0, 1)
# Add several large vertical outliers (push LS much more than LAD)
x_out <- runif(5, 2, 8)
y_out <- 2 + 0.8 * x_out + rnorm(5, 0, 1) + 60
xd <- c(x_clean, x_out)
yd <- c(y_clean, y_out)
fit_ls_d <- lm(yd ~ xd)
opt_lad_d <- optim(par = coef(fit_ls_d),
fn = function(par) sum(abs(yd - (par[1] + par[2] * xd))),
method = "Nelder-Mead",
control = list(reltol = 1e-10, maxit = 10000))
coef_lad_d <- setNames(opt_lad_d$par, c("(Intercept)", "x"))
cat("\n--- 5(d) Constructed example ---\n")
```

```
##
## --- 5(d) Constructed example ---
```

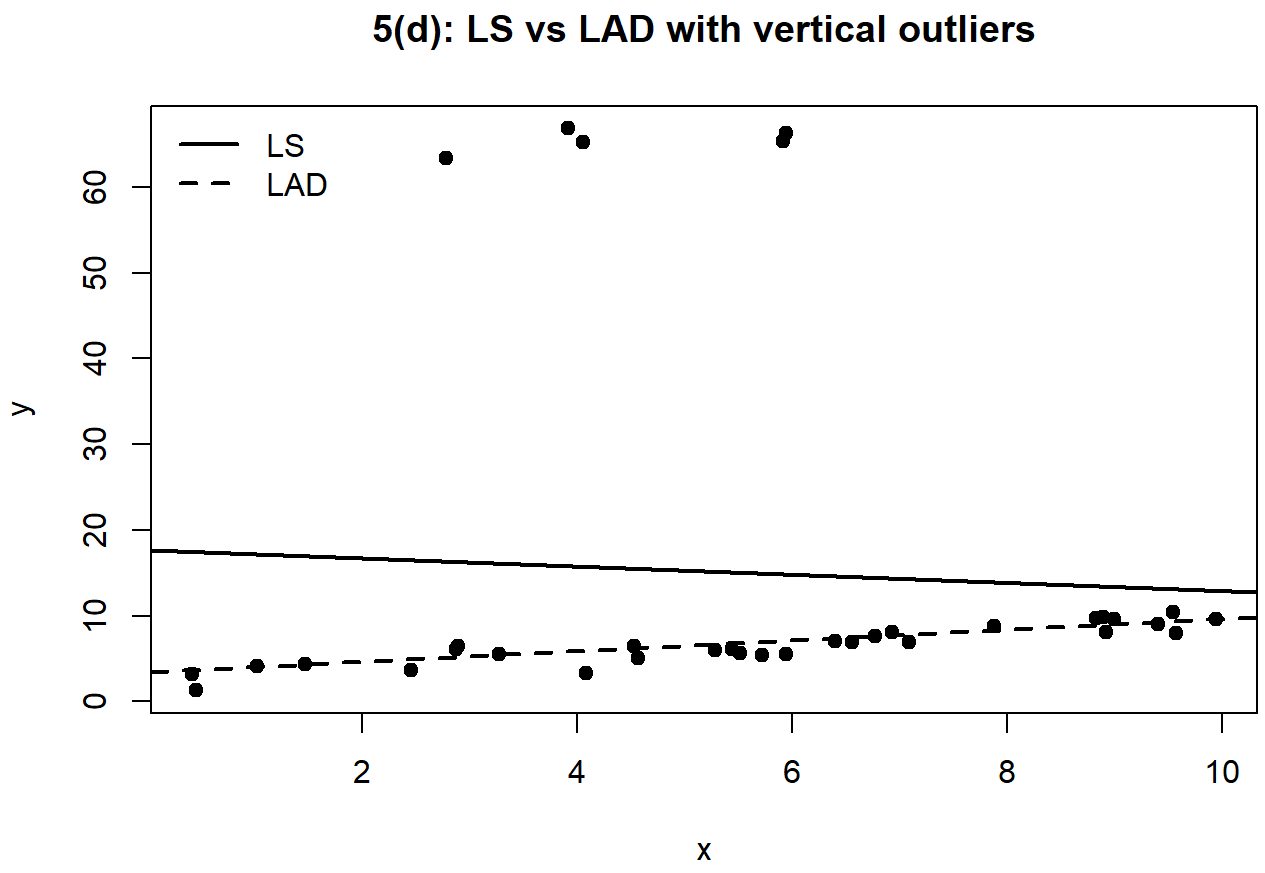
```
cat(sprintf("LS (a, b): %.3f, %.3f\n", coef(fit_ls_d)[1], coef(fit_ls_d)[2]))
```

```
## LS (a, b): 17.659, -0.477
```

```
cat(sprintf("LAD (a, b): %.3f, %.3f\n", coef_lad_d[1], coef_lad_d[2]))
```

```
## LAD (a, b): 3.434, 0.625
```

```
plot(xd, yd, pch = 19, main = "5(d): LS vs LAD with vertical outliers",
xlab = "x", ylab = "y")
abline(fit_ls_d, lwd = 2)
abline(a = coef_lad_d[1], b = coef_lad_d[2], lwd = 2, lty = 2)
legend("topleft", legend = c("LS", "LAD"), lwd = 2, lty = c(1, 2), bty = "n")
```




```
#5(e) Reverse regression (growth ~ vote) vs original (vote ~ growth)
fit_forward <- lm(y ~ x) # vote ~ growth
fit_reverse <- lm(x ~ y) # growth ~ vote
cat("\n--- 5(e) Forward (vote ~ growth) ---\n")
```

```
##
## --- 5(e) Forward (vote ~ growth) ---
```

```
##
## --- 5(e) Forward (vote ~ growth) ---

print(summary(fit_forward))
```

```
##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.9929 -0.6674  0.2556  2.3225  5.3094
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  46.2476     1.6219  28.514 8.41e-14 ***
## x             3.0605     0.6963   4.396  0.00061 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.763 on 14 degrees of freedom
## Multiple R-squared:  0.5798, Adjusted R-squared:  0.5498
## F-statistic: 19.32 on 1 and 14 DF,  p-value: 0.00061
```

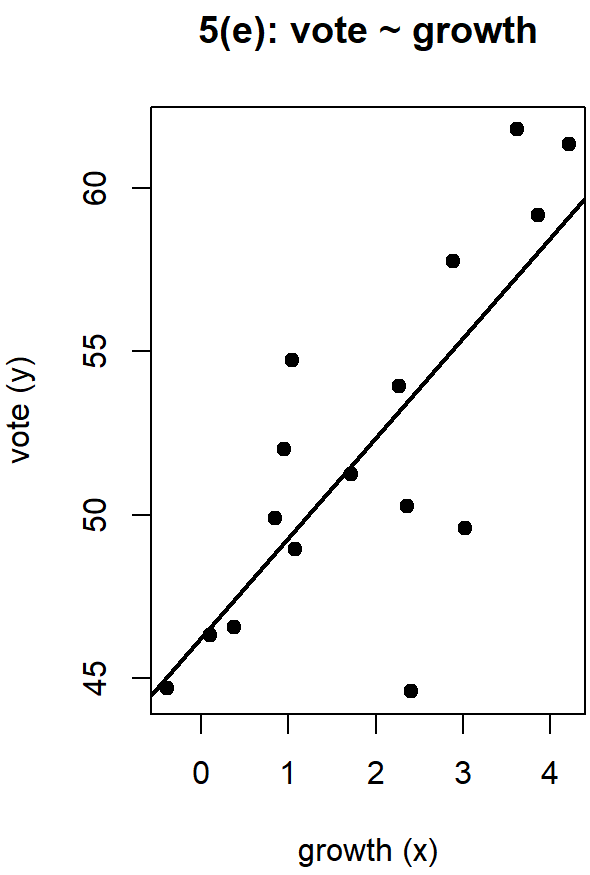
```
cat("\n--- 5(e) Reverse (growth ~ vote) ---\n")
```

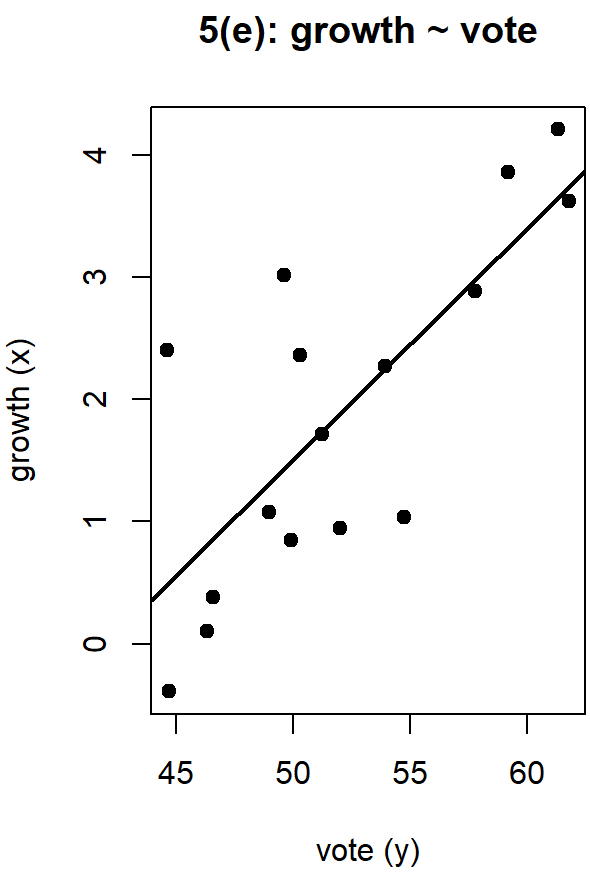
```
##
## --- 5(e) Reverse (growth ~ vote) ---
```

```
print(summary(fit_reverse))
```

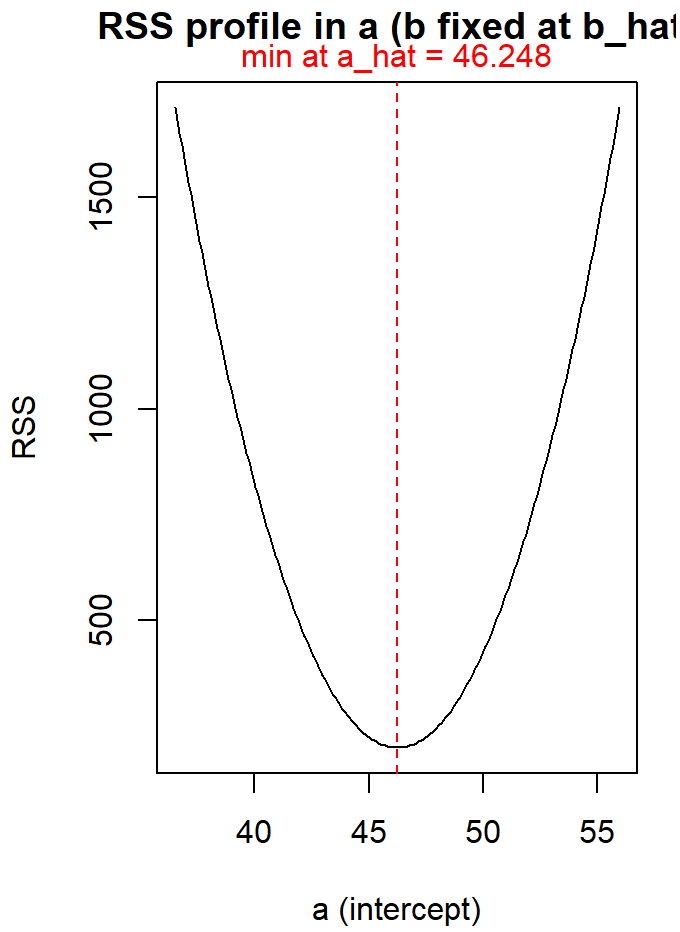
```
##
## Call:
## lm(formula = x ~ y)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.3662 -0.6586 -0.1051  0.5686  1.9149
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -7.9648     2.2559  -3.531  0.00333 **
## y             0.1895     0.0431   4.396  0.00061 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9363 on 14 degrees of freedom
## Multiple R-squared:  0.5798, Adjusted R-squared:  0.5498
## F-statistic: 19.32 on 1 and 14 DF,  p-value: 0.00061
```

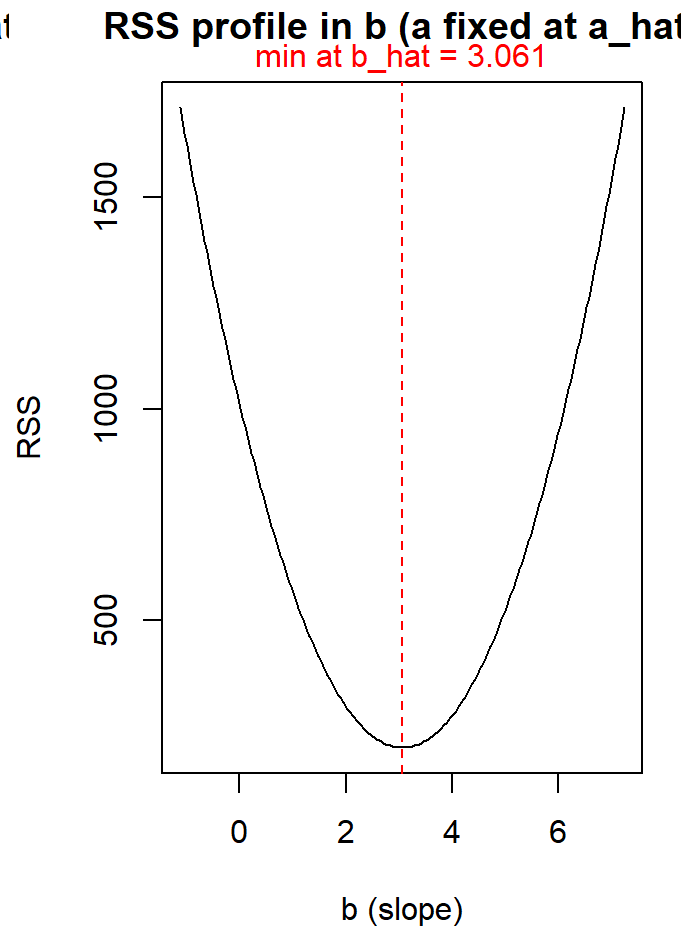
```
op <- par(mfrow = c(1, 2))
plot(x, y, pch = 19, main = "5(e): vote ~ growth",
xlab = "growth (x)", ylab = "vote (y)")
abline(fit_forward, lwd = 2)
plot(y, x, pch = 19, main = "5(e): growth ~ vote",
xlab = "vote (y)", ylab = "growth (x)")
abline(fit_reverse, lwd = 2)
```


5(e): vote ~ growth
vote (y)
45
50
55
60
0
1
2
3
4
growth (x)


5(e): growth ~ vote
growth (x)
0
1
2
3
4
45
50
55
60
vote (y)

```r
#Common setup
stopifnot(exists("hibbs"))
stopifnot(all(c("vote","growth") %in% names(hibbs)))

y <- hibbs$vote
X <- hibbs$growth

fit <- lm(vote ~ growth, data = hibbs)
a_hat <- unname(coef(fit)[1])
b_hat <- unname(coef(fit)[2])

n <- length(y)
sigma_hat_mle <- sqrt(mean(resid(fit)^2))  # uses 1/n

# Standard errors for plotting ranges
se_vec <- sqrt(diag(vcov(fit)))
se_a <- se_vec[1]; se_b <- se_vec[2]

#Part 5A

# rss(a,b): residual sum of squares at (a,b)
rss <- function(a, b, y, X) sum( (y - (a + b*X))^2 )

# Sequences around the LS estimates (wide enough to see curvature clearly)
a_seq <- seq(a_hat - 6*se_a, a_hat + 6*se_a, length.out = 200)
b_seq <- seq(b_hat - 6*se_b, b_hat + 6*se_b, length.out = 200)

# Profile RSS in 'a' with b fixed at b_hat; and in 'b' with a fixed at a_hat
rss_a <- sapply(a_seq, function(a) rss(a, b_hat, y, X))
rss_b <- sapply(b_seq, function(b) rss(a_hat, b, y, X))

op <- par(mfrow = c(1,2), mar = c(4,4,3,1))
plot(a_seq, rss_a, type = "l",
     xlab = "a (intercept)", ylab = "RSS",
     main = "RSS profile in a (b fixed at b_hat)")
abline(v = a_hat, lty = 2, col = "red")
mtext(sprintf("min at a_hat = %.3f", a_hat), side = 3, line = 0.2, col = "red")

plot(b_seq, rss_b, type = "l",
     xlab = "b (slope)", ylab = "RSS",
     main = "RSS profile in b (a fixed at a_hat)")
abline(v = b_hat, lty = 2, col = "red")
mtext(sprintf("min at b_hat = %.3f", b_hat), side = 3, line = 0.2, col = "red")
```

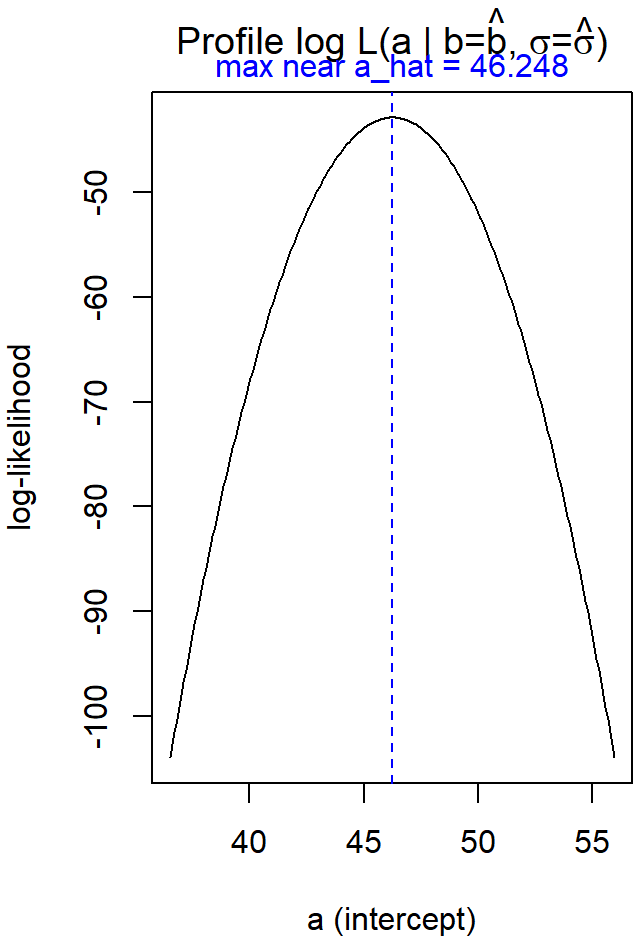


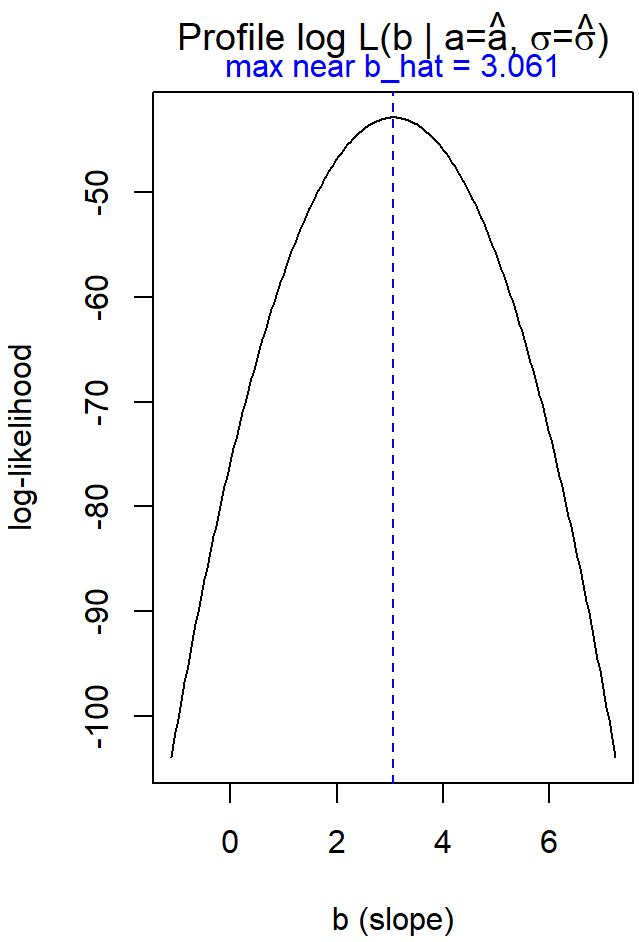



```
par(op)

# Numeric confirmation of minima
data.frame(
  a_at_min_rss = a_seq[which.min(rss_a)],
  b_at_min_rss = b_seq[which.min(rss_b)],
  a_hat = a_hat,
  b_hat = b_hat
)
```

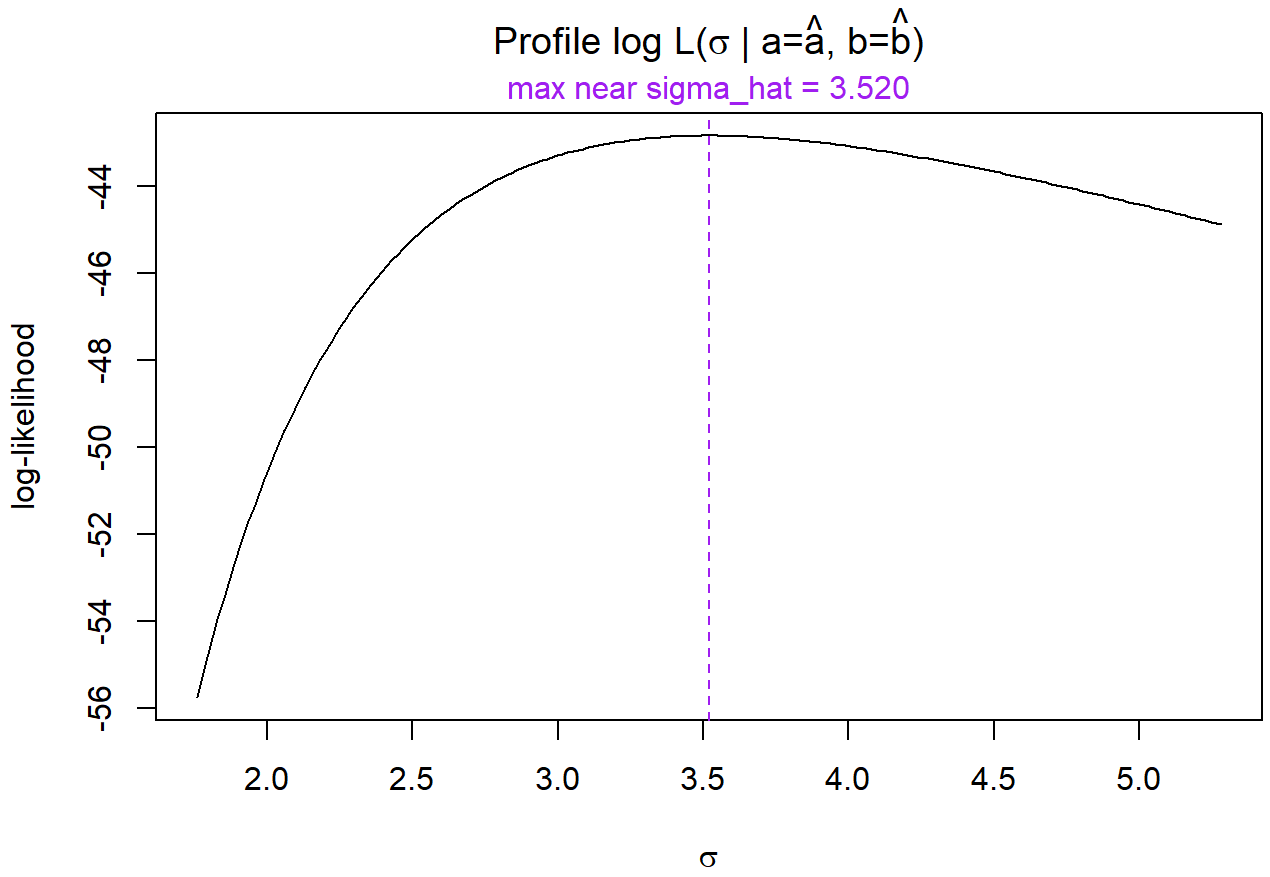
```
##   a_at_min_rss b_at_min_rss    a_hat    b_hat
## 1     46.19875     3.039535 46.24765 3.060528
```

```
#Part 5B

# Log-likelihood for Normal errors (sum over i)
loglik <- function(a, b, sigma, y, X) {
  if (!is.numeric(sigma) || length(sigma) != 1L || sigma <= 0) return(-Inf)
  sum(dnorm(y, mean = a + b*X, sd = sigma, log = TRUE))
}

sigma_use <- sigma_hat_mle  # σ fixed at MLE (1/n)

# Profiles in a and b
ll_a <- sapply(a_seq, function(a) loglik(a = a, b = b_hat, sigma = sigma_use, y = y, X = X))
ll_b <- sapply(b_seq, function(b) loglik(a = a_hat, b = b, sigma = sigma_use, y = y, X = X))

op <- par(mfrow = c(1,2), mar = c(4,4,3,1))
plot(a_seq, ll_a, type = "l",
     xlab = "a (intercept)", ylab = "log-likelihood",
     main = expression(paste("Profile log L(a | b=", hat(b), ", ", sigma, "=", hat(sigma),
")")))
abline(v = a_hat, lty = 2, col = "blue")
mtext(sprintf("max near a_hat = %.3f", a_hat), side = 3, line = 0.2, col = "blue")

plot(b_seq, ll_b, type = "l",
     xlab = "b (slope)", ylab = "log-likelihood",
     main = expression(paste("Profile log L(b | a=", hat(a), ", ", sigma, "=", hat(sigma),
")")))
abline(v = b_hat, lty = 2, col = "blue")
mtext(sprintf("max near b_hat = %.3f", b_hat), side = 3, line = 0.2, col = "blue")
```

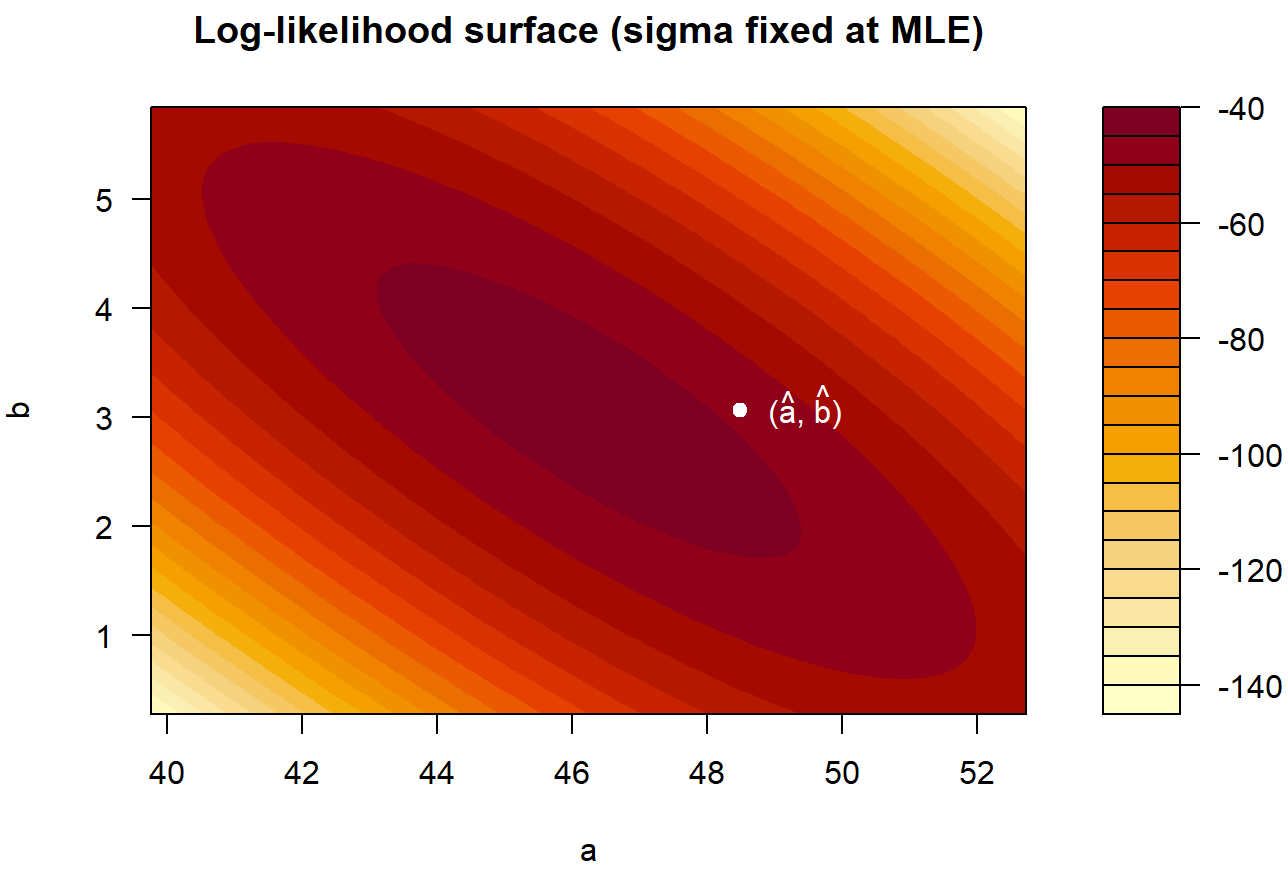





```
par(op)

# Optional: profile in sigma to match prompt literally
sig_seq <- seq(0.5 * sigma_hat_mle, 1.5 * sigma_hat_mle, length.out = 200)
ll_sig  <- sapply(sig_seq, function(s) loglik(a_hat, b_hat, s, y, X))
plot(sig_seq, ll_sig, type = "l",
     xlab = expression(sigma), ylab = "log-likelihood",
  main = expression(paste("Profile log L(", sigma, " | a=", hat(a), ", b=", hat(b), ")")))
abline(v = sigma_hat_mle, lty = 2, col = "purple")
mtext(sprintf("max near sigma_hat = %.3f", sigma_hat_mle), side = 3, line = 0.2, col = "purple")
```




```
# 2-D surface for (a,b) with σ fixed
a_grid <- seq(a_hat - 4*se_a, a_hat + 4*se_a, length.out = 80)
b_grid <- seq(b_hat - 4*se_b, b_hat + 4*se_b, length.out = 80)
LL <- outer(a_grid, b_grid, Vectorize(function(a, b) loglik(a, b, sigma_use, y, X)))
filled.contour(a_grid, b_grid, LL, xlab = "a", ylab = "b",
               main = "Log-likelihood surface (sigma fixed at MLE)")
points(a_hat, b_hat, pch = 19, col = "white")
text(a_hat, b_hat, labels = expression(paste(" (", hat(a), ", ", hat(b), ")")),
     pos = 4, col = "white")
```




```
# Numeric check of maxima on profiles
data.frame(
  a_at_max_ll = a_seq[which.max(ll_a)],
  b_at_max_ll = b_seq[which.max(ll_b)],
  a_hat = a_hat,
  b_hat = b_hat
)
```

```
##   a_at_max_ll b_at_max_ll    a_hat    b_hat
## 1    46.19875    3.039535 46.24765 3.060528
```

```
# We'll reuse our 'fourth_down' data if it already exists; otherwise, rebuild quickly.
if (!exists("fourth_down")) {
library(nflfastR)
library(dplyr)
pbp <- load_pbp(2023)
fourth_down <- pbp %>%
dplyr::filter(down == 4, !is.na(yardline_100)) %>%
dplyr::mutate(go_for_it = ifelse(play_type %in% c("run","pass"), 1, 0))
}

# Choose variables:

# x = yardline_100 (distance to opponent end zone)

# y = go_for_it (binary: 1 if team went for it on 4th down, 0 otherwise)

x <- fourth_down$yardline_100
y <- fourth_down$go_for_it

# Fit the two simple OLS regressions with intercepts

fit_y_on_x <- lm(y ~ x)  # "forward" regression y ~ x
fit_x_on_y <- lm(x ~ y)  # "reverse" regression x ~ y

# Extract the slopes (β_yx and β_xy). coef()[2] is the slope when an intercept is present.

beta_yx <- coef(fit_y_on_x)[2]
beta_xy <- coef(fit_x_on_y)[2]

# Compute sample correlation r_xy and its square

r_xy  <- cor(x, y, use = "complete.obs")
r2_xy <- r_xy^2

# Product of the two slopes

prod_betas <- as.numeric(beta_yx * beta_xy)

# Display everything together (the product should numerically equal r^2)

data.frame(
beta_yx = beta_yx,
beta_xy = beta_xy,
`beta_yx * beta_xy` = prod_betas,
`cor(x,y)^2` = r2_xy,
check_equal = all.equal(prod_betas, r2_xy, tolerance = 1e-12)
)
```

```
##        beta_yx   beta_xy beta_yx...beta_xy cor.x.y..2 check_equal
## x -0.003359244 -13.32692        0.04476838 0.04476838        TRUE
```

Conclusion:The analysis confirms that the product of the forward and reverse regression slopes equals the square of the correlation between x and y, which is $\beta_{yx}\beta_{xy} = r^2$. Since $|r| \leq 1$, this product is always less than or equal to 1. In our dataset, $\beta_{yx}\beta_{xy} = 0.0448$, which matches $r^2 = 0.0448$, verifying the relationship. This small value indicates a weak but slightly negative relationship between field position and the decision to "go for it" on 4th down.